Last modified on

Sports Data Soccer API - Remote Pressure Timeline Feed (MA41)

Get aggregated match pressure timeline data during the course of a match, based on “remote” data

Overview

The Soccer Remote Pressure Timeline feed provides the timeline of aggregated pressure metrics during the course of a match, based on “remote” data, along with contestants, goals, cards, substitutes and match official's detail.

> **Required Compliance:**
> To comply with our API security and best practice logic for your API calls please refer to our FAQs. We also recommend that you take a look at our API Overview and Authentication Guide. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Contact your Stats Perform representative for a full list of available comps.

UPDATED	POLLING FREQUENCY							
	TIER 15	TIER 12+14	TIER 4	TIER 13	TIER 8+9+10+11	TIER 6+7	TIER 5	TIER 1+2+3
Updated on every stop in play during the game, including for the following types of events: goal, shot at goal, assist, free kick, offside given, cards, start of the half, end of the half, corner, substitution, lineup **Note:** There will always be a gap of at least 30 seconds between files unless a goal is scored	See Tier 13	Post-match	n/a	Post-match	n/a	n/a	n/a	n/a

---

Remote Pressure Timeline (MA41) Guide

USAGE	FEED
GET **remote pressure timeline** for the match based on remotely collected data.	`/soccerdata/remotepressuretimeline/{outle` `{queryParameters}`

> If your request URL is as follows (does not contain a mandatory fixture UUID), the feed returns an error:
> `/soccerdata/remotepressuretimeline/{outletAuthKey}`

Query Parameters And Example Request Parameters

This feed also makes use of the Global query parameters, and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you must query the `/remotepressuretimeline` feed by a fixture (match) UUID.

Legacy IDs: You can use the `fx` query parameter to filter by an Opta legacy ID (only one ID is supported - you cannot specify a comma-separated list of IDs).

> **How to use query parameters and make successful requests:**
> In our guides, we use HTTPS syntax for our example `GET` requests (rather than cURL, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You MUST format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the main guide and API Overview and Authorisation guide.
>
> - Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` this must be after the Outlet Authentication Key (and endpoint, if applicable). for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format), entity query parameter and UUID.
> - Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or - if a parameter supports it - `,` (multiple values, 'OR') or `!` ('NOT').
> - Use the required `_` (underscore) for any parameters listed with this prefix.
> - Remove any placeholder value curly braces `{}` from your calls (we include these as an example only).
> - If using the JSONP format (`_fmt=jsonp`) you MUST use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Parameter	Value and description
`fixtureUuid`	GET remote Pressure Timeline list by **fixture UUID** inline URL path.

Parameter	Value and description
	`GET https://api.performfeeds.com/soccerdata/remotepressuretimeline/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}`
`fx`	GET remote Aggregated Events list by **fixture UUID**. Pass the fixture/match UUID to the `fx` parameter. `GET https://api.performfeeds.com/soccerdata/remotepressuretimeline/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&fx={fixtureUuid}`

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- XML
- JSON

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

▸<remotePressureTimeline> (XML only)

▸<matchInfo>

▸<description>

▸<sport>

▸<ruleset>

▸<competition>

▸<country>

▸<tournamentCalendar>

▸<stage>

▸<contestants> (XML only)

▸<contestant>

▸<country>

▸<venue>

▸<liveData>

▸<matchDetails>

▸<periods> (XML only)

▸<period>

▸<suspensions>

▸<suspension>

▸<scores>

▸<ht>

▸<ft>

▸<total>

▸<pressureInfo>

▸<pressure>

▸<goal>

▸<missedPen>

▸<card>

▸<substitute>

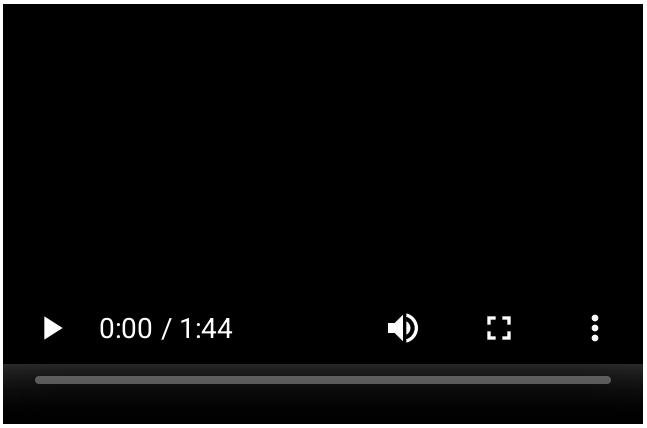
▸<matchDetailsExtra>

▸<matchOfficials> (XML only)

▸<matchOfficial>

Pressure Intensity

To improve a player's passing ability, it's important to understand the level of pressure they are under in their position. The new Opta Vision metrics automatically detects when the defending team approaches the player in position with the goal of winning the ball back, limiting their passing options. Please watch the video below to learn how these new measures work.



Statistic Types

Click on the buttons below to see the complete lists of statistics names and their description

▸**Statistics - Names and Descriptions**